

IronSign - Technical Documentation

English translation

1. Introduction

A distributed blockchain transaction signing system should, in a very simplified form, work as follows: the system takes an unsigned transaction as input and provides a transaction signed as if it had been signed by a single private key, while in reality the signature is created in a distributed manner.

The demonstration prototype implemented for this project presents a reduced model of one possible scenario for how such a system could be implemented in practice for custodial solutions such as Fireblocks, using open-source technologies and libraries. The project is demonstrated using a local simulated environment, while taking into account that every component implemented or used in the prototype, as well as the design choices made here, could be abstracted and implemented in a different, production-ready manner in a real production environment. Where relevant, we explicitly point out how a particular part of the project could potentially be replaced by a production-grade variant.

2. Research

The motivation for the project was the effort to achieve a level of security and practical usability in digital-asset private-key management comparable to commercial solutions based on multi-party computation (MPC), typically represented by the Fireblocks platform. Such systems make it possible to sign transactions without any single entity ever possessing the complete private key, while also providing integration with custodial service infrastructure. A key motivation was therefore to create an open, auditable, and extensible alternative suitable for internal deployment, with a foundation that could also be used in a prototype application.

A significant part of the project consisted of researching available protocols and existing implementations. At the beginning of the work, it was not clear which protocols were practically usable, which open-source implementations existed, which blockchains were compatible in terms of their signature schemes, or what would have to be implemented beyond the existing libraries. For this reason, the research was conducted systematically and accounted for a substantial part of the overall project effort (estimated at roughly half of the time spent on the project). The result of the research was not only the selection of a particular library, but also a set of technical decisions that subsequently influenced the architecture and implementation design according to the selected library.

[1] <https://www.fireblocks.com/>

[2] <https://www.fireblocks.com/what-is-mpc>

In the initial phase of the research, we focused on the theoretical background of MPC protocols and the cryptographic primitives used in this domain. It gradually became clear that, from a practical perspective, threshold signatures (TSS) best matched our requirements. TSS makes it possible to create a signature equivalent to a standard single-signature while the signature itself is produced through cooperation between multiple parties. Threshold signatures are commonly used in (t,n) -threshold systems, meaning that t out of n parties must cooperate in order to produce a valid signature. In our case, however, so-called "non-threshold signatures" are sufficient; these are threshold signatures configured as an (n,n) -threshold system. One of the main reasons why (n,n) -threshold is sufficient for us is that the system is operated under the custodian's control, so possible outages are also under our control. This is discussed further in the Architecture chapter.

Because the project initially targeted support for Bitcoin and Ethereum, a key criterion was finding a signature scheme shared by both ecosystems and enabling multichain use without major changes to the cryptographic core. An analysis of blockchain compatibility led to the selection of ECDSA over the secp256k1 curve as the natural intersection: Ethereum uses ECDSA for transaction signatures, while Bitcoin uses it for legacy and SegWit addresses. This choice provides native support for both BTC and ETH without changing the signature scheme between chains.

Taproot was considered as well. It is possible to send funds from legacy/SegWit addresses to Taproot addresses. The only relevant limitation for our use case is that Taproot outputs cannot be spent using ECDSA, because Bitcoin uses Schnorr signatures there. This limitation is not a major issue in the custodial model: the system can keep funds in legacy/SegWit addresses under its own control while still sending funds to customers' Taproot addresses without restrictions; spending from those Taproot addresses remains the user's responsibility.

We then surveyed existing MPC/TSS implementations in different programming languages and with different properties. Relevant examples included:

1. open-source implementations and libraries associated with Fireblocks (mpc-lib / MPC-CMP)
2. the TaurusHQ implementation in Go (taurusHQ/multi-party-sig)
3. the Binance implementation (bnb-chain/tss-lib)
4. many other academic or community implementations of threshold ECDSA/Schnorr, including libraries derived from CGGMP21/CGGMP24, implementations from the Dfns ecosystem, and standalone FROST implementations for Schnorr signatures

[3] https://en.wikipedia.org/wiki/Threshold_cryptosystem

We selected an implementation in Rust primarily because of its emphasis on security (type safety and memory safety), better integration with existing internal systems, and suitability for future audits. Another practical factor was communication with the maintainers of the selected solution, which indicated active library development and a higher likelihood of long-term sustainability.

As a result of the research, we decided to use an implementation of the CGGMP family of protocols, specifically the variant referred to as CGGMP24, available as a branch in the LFDT-Lockness/cggmp21 repository. CGGMP24 is a newer iteration closely related to CGGMP21, but it addresses identified security aspects (<https://github.com/LFDT-Lockness/cggmp21/security>) and modifies some parts of the protocol. From the project perspective, it provides a suitable compromise between practical usability, security properties, and availability as an open-source implementation in Rust.

The final step of the research was to determine whether any open-source projects already built a higher-level layer on top of CGGMP24, such as a wallet-like system, so that the project could move towards the target type of solution without implementing everything from scratch. We identified only one existing demo, namely `m2ux/tss_demo`, which demonstrates basic use of the library in a real communication setting. This demo provided useful information for understanding practical integration and message flows, but its parameters and functionality did not match the requirements of our project, including its different use case and the lack of extensions such as presignature generation and an offline signing node. This led us to design and implement our own solution on top of CGGMP24: simple, understandable, and extensible, with a demonstration application serving as an open foundation for MPC/TSS signing in a custodial environment.

2.1. Presignatures

During the evaluation of the library, we identified that CGGMP24 supports presignatures, which make it possible to split threshold signing into two phases: precomputation and final message signing. A presignature is a cryptographic object in which all random values and parameters required for a future signature are generated and fixed in advance, without knowing the particular message that will be signed. As a result, each party receives its own local presignature record, intended for one-time use, while corresponding public auxiliary data is associated with the presignature set and later allows the final signature to be correctly verified and combined.

When a message later becomes available for signing, the expensive protocol does not need to be executed again. Each party can deterministically compute a partial signature from its presignature record and the message. The partial signatures can then be aggregated into a standard ECDSA signature. The practical effect is that the expensive part of the work - communication, randomness, commitments, and so on - is moved earlier, while the actual signing step becomes a lightweight local operation followed by collecting and combining the results.

Presignatures also introduce an important security rule: they are strictly single-use. Reusing the same presignature set to sign different messages can leak information about the secret key, analogous to the classic ECDSA problem of reusing a nonce. The system therefore has to manage presignatures as a consumable

resource (a pool) and guarantee that each set is used at most once. The presence of presignatures in the library proved to be a key factor in the design of the prototype and its overall architecture.

3. Model Concept

Because commercial MPC solutions for blockchain transaction signing are generally closed-source, publicly available information is not sufficient to describe their internal architecture precisely. The model proposed in this project therefore represents only one possible way in which MPC signing could be integrated into a custodial system. The purpose of this chapter is to outline the data flow and the roles of the components, while also showing how the MPC/TSS model differs from traditional signing with a single private key. For simplicity, details such as verification, request validation, authorization, and other security mechanisms are omitted here; these aspects would of course have to be present in a real production environment.

3.1. Single Private Key Model

In the traditional model, signing is centralized. An internal user or client creates a transaction request. The system constructs the transaction in the format of the relevant blockchain (for example Bitcoin or Ethereum), prepares the data to be signed, and passes it to a component that possesses the private key. This component signs the transaction and the signed transaction is sent to the blockchain network.

The disadvantage of this approach is centralized risk: the complete private key must exist within a single security boundary (a process, machine, or device). This point naturally becomes a single point of failure - compromising the environment that stores and uses the key allows an attacker to create valid signatures without cooperation from other parties.



Figure 1: Single private key

3.2. MPC/TSS Network Model

In the proposed model, the data flow before signing remains the same: the transaction is created and prepared in the format of the particular blockchain. The difference occurs at the signing step. Instead of a single node holding the complete private key, the system uses a network of N MPC nodes, where each node holds only its share of the secret (key share) and can contribute to producing the final signature. The component that receives signing requests is called the coordinator. The coordinator distributes the signing request to the MPC

network, requests partial signatures from the individual nodes, and then combines them into the final signature, which is indistinguishable from a standard single-signature to the blockchain.

The main advantage is that the complete private key never appears on a single device. Compromising one part of the system is therefore not sufficient to create a valid signature; an attacker would have to defeat multiple independent components and obtain a sufficient number of shares to influence the signing process.

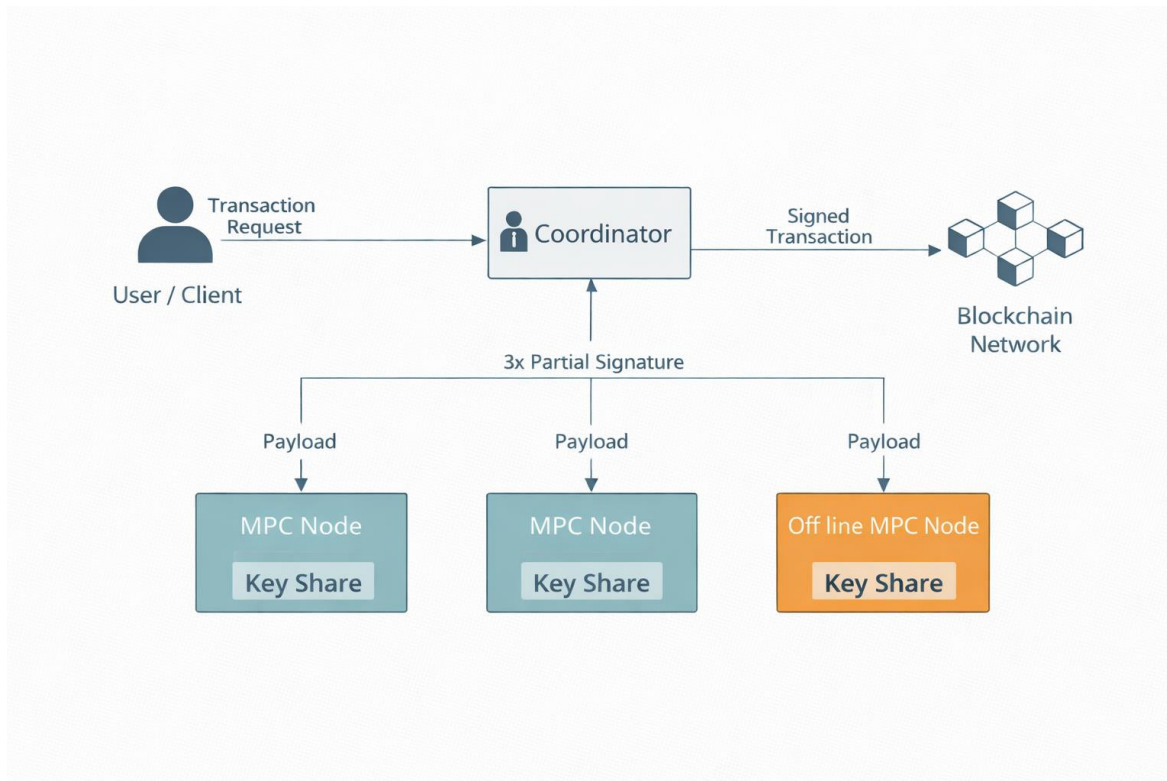


Figure 2: MPC/TSS model

3.3. Offline Node as an Additional Security Layer

To bring the architecture closer to practical custodial scenarios with higher security requirements, the model is further extended with an offline node (air-gapped signer). This node is part of the MPC network in terms of its key share, but during normal operation it is not connected to the Internet or to the internal network. The coordinator therefore communicates directly only with the online nodes ($n-1$), but needs the contribution from the offline node to construct the final signature.

In practice, this means that the transaction to be signed, or the data to be signed, is transferred to the offline node through a physical or isolated channel, such as a QR code or USB medium. The offline node performs the local computation needed to create its partial signature, and the result is transferred back to the online environment through the same offline channel. The coordinator then combines the missing partial signature with the others and constructs the final transaction signature, after which the transaction can be sent to the blockchain. This design increases resilience against compromise of the online infrastructure because part of the signing process requires physical interaction with an isolated environment. More precise details regarding the security rationale are discussed in the Threat Model chapter.

4. Architecture

4.1. Higher-level Architecture

To achieve this model in a reasonable and extensible form, the architecture is naturally divided into at least two layers:

5. an auxiliary layer responsible for transforming between the formats of specific blockchain transactions and the data to be signed
6. the MPC system itself, which performs distributed key generation and signature creation

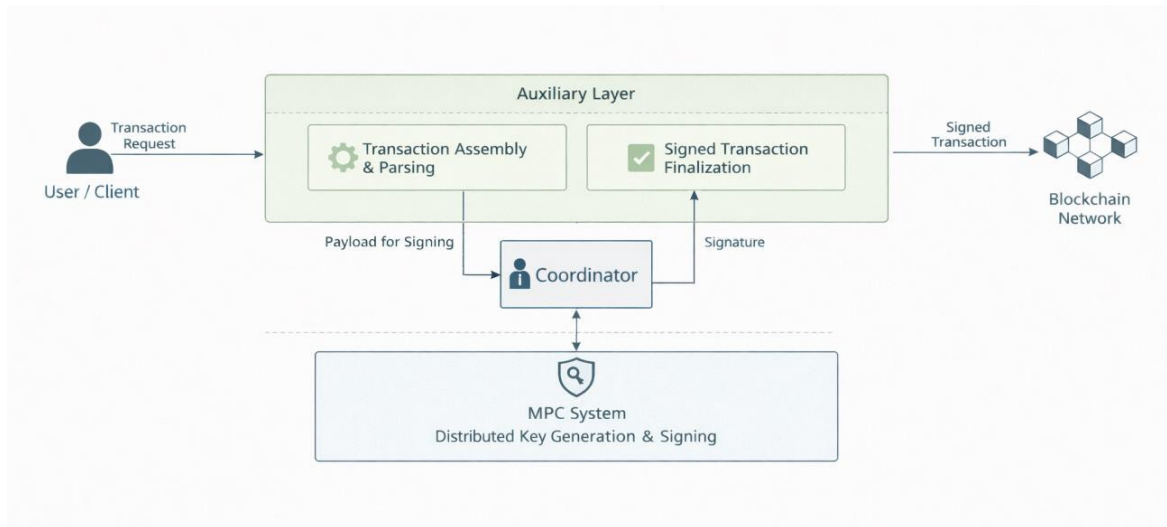


Figure 3: Higher-level architecture with auxiliary layer

It is important to emphasize that the MPC/TSS system is, in principle, only a signing black box. At its interface, it behaves similarly to local signing with a private key: it receives data to be signed and returns a signature. The MPC protocol itself does not interpret these data - it does not know whether it is a Bitcoin transaction, an Ethereum transaction, or an arbitrary sequence of bytes. Responsibility for ensuring that the correct payload is signed (for example, the transaction hash according to the rules of a given blockchain) and that the resulting signature is inserted back into the transaction correctly belongs to the auxiliary layer. This separation makes it possible to keep the MPC core simple, formally understandable, and reusable across multiple blockchains.

The auxiliary layers therefore handle tasks such as constructing or parsing a transaction, computing the exact data to be signed (including all rules specific to a given chain), passing those data to the MPC system, and subsequently finalizing the transaction by attaching the resulting signature in the correct format. From the perspective of a production deployment, this is an essential part of the system because it is what guarantees that the signature corresponds to a real and valid transaction on the given blockchain. At the same time, it is largely a separate problem - the domain logic of transaction formats - that can be isolated from the core MPC logic and implemented incrementally later.

4.2. Auxiliary Layer

The auxiliary layer is the system layer that operates at the blockchain transaction level and connects the domain logic of a specific chain with the MPC signing core. In the final project, this layer was implemented and is used to demonstrate the functionality of the entire prototype on a local Bitcoin blockchain in regtest mode.

In the current implementation, the auxiliary layer constructs a specific type of Bitcoin transaction (SegWit), prepares the correct data to be signed from it, and passes those data to the MPC system. After obtaining the resulting signature, it finalizes the transaction and sends it to the local blockchain environment. The layer also handles interaction with the regtest environment so that the complete end-to-end flow can be demonstrated: from initializing the local blockchain, through creating the transaction and signing it with the MPC system, to the blockchain accepting the transaction. Further implementation details of this layer and the regtest environment are discussed in the Implementation chapter.

From an architectural perspective, the auxiliary layer is also the bearer of multichain support. The MPC system itself signs only input data and does not distinguish whether they belong to Bitcoin, Ethereum, or another format. The responsibility of the auxiliary layer is to prepare the correct signing payload for a particular blockchain and, after obtaining the signature, construct a valid transaction for that chain. In our case, this means that the same MPC core can be used for blockchains using ECDSA over the secp256k1 curve, while the differences between individual chains are isolated in this auxiliary layer.

4.3. MPC System

The MPC system in this project is a network of signing nodes that jointly perform threshold signing without the complete private key ever appearing on a single device. Unlike a classical online MPC model, where every signature requires interactive communication between the nodes, the prototype is designed to minimize interactivity in the runtime environment and maximize the practical demonstrability of the offline node.

It is precisely the presignatures that make this design possible and are the main reason why the prototype architecture is divided into two distinct phases: an initialization phase in a controlled environment and a runtime phase in a separated environment. During initialization, all expensive computations take place and portable sets of presignatures are produced. During runtime, the nodes merely consume the prepared presignatures and locally compute partial signatures.

For demonstration and development, a minimized in-memory delivery model is used, with simple message passing through a local channel. The reason is that any specific networking implementation used in the prototype (for example, a WebSocket server) would still not meet production requirements such as authentication, authorization, encryption, monitoring, retry/backpressure, and resilience to failures. As a result, that transport layer would have to be redesigned for real deployment anyway. The minimalist in-memory model therefore makes it possible to focus time and complexity on the core of the project - the design and implementation of node logic, protocols, and supporting mechanisms - without unnecessarily getting stuck on infrastructure details.

As already mentioned in the Introduction, the implementation was designed so that node logic could be transferred to real deployment as easily as possible. This means that nodes are implemented so their behavior and lifecycle are analogous to real components: in production, nodes could run as separate processes on physical machines, VMs, containers, or other isolated runtime environments. In the prototype, nodes are represented by asynchronous Tokio tasks that communicate with the coordinator through in-memory delivery. Ignoring network outages and transport-level errors, this model reflects the behavior of a production deployment well: nodes receive messages, update their internal state, and send further messages.

[4] <https://docs.rs/tokio/latest/tokio/task/>

4.3.1. Initialization Phase

The initialization phase takes place in a controlled (isolated) environment where all signing nodes are available simultaneously and can communicate with one another. The purpose is to perform all cryptographically sensitive and communication-intensive operations in an environment for which there is a high degree of confidence in device security. This reduces the risk of compromise at the moment when key shares and presignatures are created to practically zero.

The main steps in this phase are:

- generation of auxiliary information for the protocol (auxiliary information)
- distributed key generation, resulting in key shares for the individual nodes
- pre-generation of presignatures for future signing

The output of the initialization phase consists of serialized states that can be transferred to a separate runtime environment. In the prototype implementation, this transfer is performed through files (for example, using a USB medium); architecturally, this is a general export/import mechanism, so the same model could use another transfer channel.

Importantly, the initialization phase directly produces presignature sets that are ready for signing in the runtime environment. As a result, the runtime environment does not need to execute complex protocols when signing transactions.

4.3.2. Runtime Environment (Coordinator + Online Nodes)

The runtime phase runs in a separate environment representing a scenario closer to production. This environment contains:

- a coordinator that receives signing requests from the auxiliary layer and controls the signing flow
- n-1 online nodes connected to the coordinator and holding local presignature pools imported from the initialization phase

At runtime, the nodes no longer need to execute the interactive MPC protocol. After receiving a signing request, each online node:

- selects the next presignature from its pool
- locally computes a partial signature for the given data
- sends the partial signature to the coordinator

A security requirement is that a presignature must never be used more than once. In the prototype, presignatures are therefore managed as a FIFO queue and removed from the pool after use. The coordinator collects partial signatures from the online nodes, but the finalization of the signature still requires the contribution from the offline node.

4.3.3. Offline Node (Air-gapped Signer)

The prototype also includes a separate program representing the offline node. This node is deployed on separate hardware and does not communicate over the network with the coordinator or online nodes during runtime. The offline node imports its own presignature pool from the same initialization phase.

When a message must be signed, the data to be signed are transferred to the offline node through an offline channel, implemented in the prototype as a file on a USB medium. The offline node consumes one presignature from its pool, computes a partial signature, and transfers the result back to the runtime environment through the same offline channel. The coordinator then supplies the missing partial signature and performs the final aggregation.

The chosen transfer channel (files/USB) is a practical approximation of offline transfer in the prototype; the same concept is compatible with alternatives such as QR codes.

4.3.4. Signature Aggregation

After obtaining all required partial signatures, the coordinator aggregates them into the final signature. The aggregation uses the public data associated with the presignature set (commitments), which are the same for all participants in that set and allow the signature to be correctly combined and subsequently verified. The resulting signature is passed to the auxiliary layer, which inserts it into the transaction and finalizes the transaction.

4.3.5. Prototype Limitations

The prototype is designed as a demonstration system and does not primarily address the following aspects:

- authentication and authorization of requests to the coordinator and nodes
- encryption and transport integrity for online communication
- handling node failures, network errors, timeouts, and repeated delivery
- restarts, rollbacks, and desynchronization of presignature pools

5. Implementation

This chapter focuses on the practical realization of the proposed system. The goal is not to comment on individual blocks of source code in detail, but rather to explain the key implementation decisions, the project structure, and the parts of the solution that are technically most important or interesting. We assume that the source code is available to the reader and, given its structure and comments, allows further details to be filled in independently without being discussed explicitly here.

The chapter is therefore divided into several thematic subsections. We first briefly describe the project structure and division into components, and then discuss the individual implementation components.

5.1. Structure

From an implementation perspective, the project is divided into three main parts. The first is the MPC system itself, implemented in three separate modules corresponding to the three execution modes of the program: `coord_runtime`, `initialization`, and `offline_node`. Together, these modules cover the entire lifecycle of the system - from preparing states in a controlled environment, through the online runtime environment with a coordinator, to the standalone offline signing node.

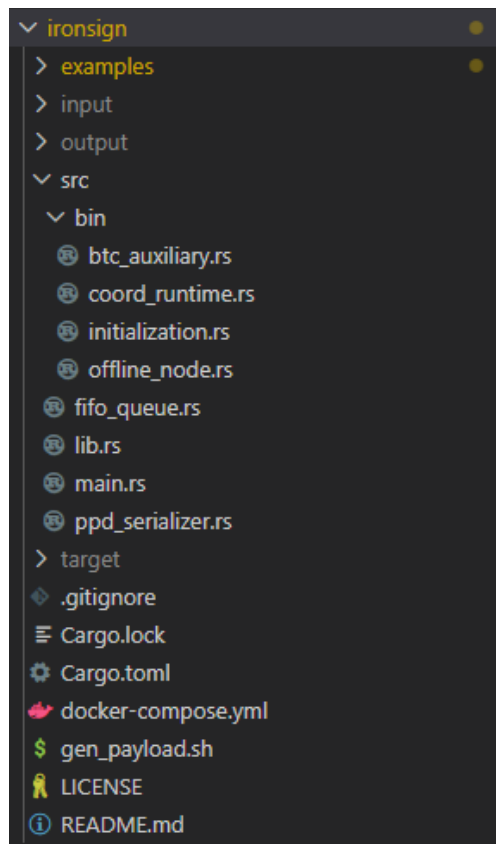


Figure 4: Compressed flowchart - Initialization

The second part is the `btc_auxiliary` module, which represents the Bitcoin auxiliary layer. Its task is to work with real Bitcoin transactions in a regtest environment: construct transactions, prepare the signing payload for the MPC system, finalize the transaction after receiving the signature, and interact with the local blockchain environment. This module therefore connects abstract MPC signing with a concrete use case at the Bitcoin transaction level.

The third part consists of supporting implementation modules. The `fifo_queue` module manages presignatures as a queue. Although Rust provides data structures suitable for FIFO behavior, such as `VecDeque`, the project implements its own simplified abstraction that limits queue operations to those matching the security model for working with presignatures. The goal is to minimize the possibility of incorrect manipulation of the presignature pool and to express more clearly that presignatures must be consumed strictly once and in the expected order.

Another supporting module is `ppd_serializer`, which handles serialization of the `PresignaturePublicData` structure. This structure is required by the `CGGMP24` library for aggregating partial signatures into a final signature, but the library itself does not directly provide `Serialize` and `Deserialize` implementations for it. A custom serialization mechanism was therefore required so these data could be correctly stored, transferred, and loaded again within the proposed architecture.

In addition to the source modules, the repository contains other supporting files. The `examples` directory contains experimental and test programs created during development to better understand the behavior of the `CGGMP24` library and validate individual design decisions. The `gen_payload.sh` script was used in earlier stages to generate a random payload for testing signatures; after implementing `btc_auxiliary`, it is no longer necessary to work only with artificial payloads because the system can generate real signing data derived from a Bitcoin transaction. The `docker-compose.yml` file is used to start a local Bitcoin blockchain in regtest mode as a Docker container, providing an isolated and reproducible test environment for the end-to-end demonstration.

5.2. Regtest

One of the project goals was not only to show that the MPC system can correctly sign arbitrary input data, but also that it can correctly sign the data it is actually intended to sign in practice. In our case, these are signing payloads derived from Bitcoin transactions. Testing on the real Bitcoin blockchain would have been unnecessarily demanding for the project, more difficult to manage, and would have required real funds. We therefore chose to use a local Bitcoin blockchain in regtest mode.

Regtest is a testing environment in which a Bitcoin blockchain can be run locally and fully controlled through commands. This makes it possible, for example, to create new addresses, assign funds to them, generate any number of blocks, and repeatedly test scenarios without depending on the public network. For this project, it is an ideal solution because it allows the system's behavior to be verified on real Bitcoin transactions while removing the practical complications of testing on a production blockchain.

Using regtest allows us to demonstrate the end-to-end functionality of the entire solution: from creating the transaction, through preparing the signing payload and signing it with the MPC system, to finalizing and accepting the transaction on the blockchain. In the implementation, the regtest environment is started locally as a Docker container, which also simplifies the reproducibility of experiments and use of the project on different machines.

5.3. BTC Auxiliary

Looking at Figure 3, the auxiliary layer should ideally handle transaction-related tasks: transaction construction, preparation of the signing payload, finalization after obtaining the signature, and communication with the MPC system through the coordinator. In a production environment, this layer would naturally handle only the domain logic of Bitcoin transactions and integration with the signing core. The blockchain itself would operate independently of the system and would not be directly managed by the application.

In the demonstration prototype, however, `btc_auxiliary` has somewhat broader responsibilities. In addition to the tasks above, it also handles the local regtest blockchain, including address creation, block generation, and initialization of test conditions. The reason is to simplify the demonstration: instead of requiring manual blockchain management or another separate program, all logic related to the Bitcoin regtest environment is centralized in one auxiliary layer. Since this layer already works with the Bitcoin API and transaction formats, this is a natural compromise that simplifies the overall control flow without changing the architectural essence of the MPC system.

From the point of view of the functional demonstration, it is important to show that the system does not only sign arbitrary data, but also correctly handles a real Bitcoin transaction. Consider addresses A, B, and C, where address B is derived from a public key provided by the MPC system. In the regtest environment, funds are first assigned to address A, and bitcoins are then sent from A to B. This produces a UTXO linked to the public key of the MPC system.

Next, a new transaction is constructed to send funds from address B to address C. The blockchain can accept this transaction only if it contains a valid signature corresponding to the public key of address B. Because address B was derived from the public key of the MPC system, successfully signing such a transaction demonstrates that the MPC system effectively possesses signing capability corresponding to the private key for that address.

The role of `btc_auxiliary` in this process is to prepare the correct signing payload from the transaction, pass it to the MPC system for signing, insert the resulting signature back into the transaction, and then send the transaction to the blockchain environment. Finally, it generates additional blocks so that the transaction becomes confirmed and its acceptance can be verified unambiguously. If the blockchain accepts the transaction, the demonstration confirms not only the correctness of the signature itself, but also that the entire end-to-end flow - from transaction creation, through MPC signing, to finalization and submission to the network - works correctly for a real Bitcoin scenario.

5.4. Initialization

The initialization phase is a separate part of the system whose task is to prepare all data required for subsequent signing in the runtime environment. In the classical use of the CGGMP protocol, individual nodes

would communicate over a network, pass through interactive protocol phases, and gradually generate auxiliary data, key shares, and later presignatures in a distributed fashion. During the design of the project, we initially started from exactly this assumption and considered that we would need to create at least a local mini-network of nodes that would communicate with one another and route messages to simulate real distributed operation.

It gradually became clear, however, that such a model was not necessary for our particular use case. Since the initialization phase runs in a controlled and trusted environment where device security can be guaranteed, there is no need to build either a network or a local communication layer between nodes merely to obtain the resulting states needed for later signing. A key discovery was that the round-based library on which CGGMP24 is built provides a simulation mechanism, `round_based::sim`. This mechanism makes it possible to locally simulate multiple protocol parties on a single device and obtain exactly the same resulting states as would be produced by a real distributed execution.

In practice, this means that the initialization phase does not need any networking or local message routing. Instead, a simulated execution for all nodes is run on one device at once, producing the individual state of each node. This significantly reduces implementation complexity and the number of lines of code, while leaving the cryptographic meaning of the result unchanged. From the perspective of the project, this was one of the most important design decisions because it enabled the expensive initialization logic to be separated from runtime signing and made the overall model considerably simpler and more practical.

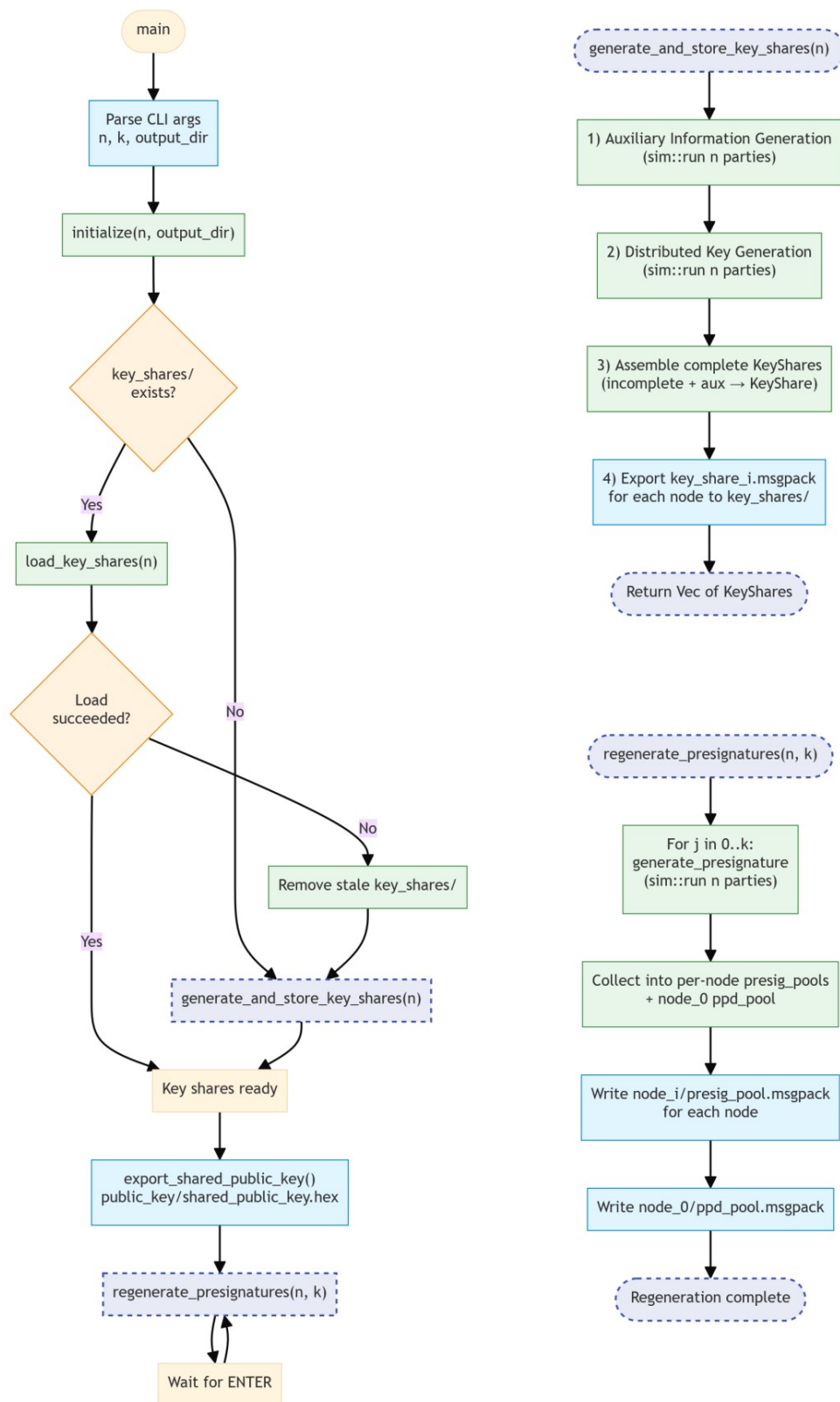


Figure 5: Compressed flowchart - Initialization

From a cryptographic perspective, initialization proceeds through several steps. First, auxiliary information (aux_info) is generated for each party. Key generation then takes place, producing so-called incomplete_keyshare values. Only by combining aux_info with incomplete_keyshare is the final key_share

created. The `key_share` can then be used to generate presignatures that are later consumed during signing. Importantly, once a final `key_share` has been stored, `aux_info` and `incomplete_keyshare` do not need to be generated again; when initialization is run again, the existing `key_share` can simply be loaded and used as the basis for generating a new presignature pool.

This also has a practical consequence for state persistence. For runtime signing itself, it is sufficient for each node to have only its `presig_pool`. For system recovery or generation of additional presignatures, however, the `key_share` must be retained. Our design therefore creates and stores both types of data, but for different purposes: presignatures are intended for transfer to the runtime environment and the offline node, while key shares serve as the basis for future pool replenishment and as a backup.

Implementation-wise, a separate `node_i` directory is created for each node, and its local `presig_pool` is serialized into that directory. In our model, `node_0` is considered the offline node, so additional public data required for later signature aggregation, specifically `PresignaturePublicData` (referred to as `ppd` in the project), are prepared for it. The final `key_share` values are stored separately in a `key_shares` directory. On subsequent runs of the initialization phase, the system first checks whether the `key_shares` directory exists and contains usable keys. If so, it reuses them and generates only a new presignature pool. Otherwise, the entire process is started from the beginning.

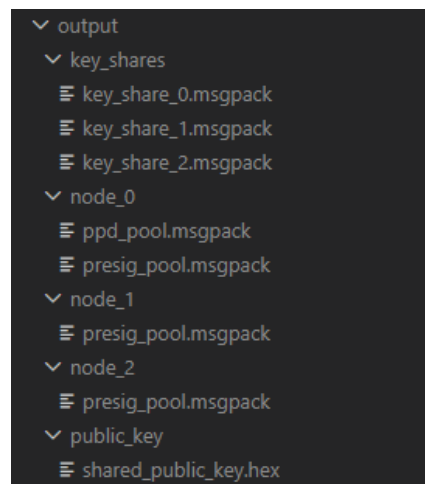


Figure 6: Example Initialization output for 3 nodes

The result of the initialization phase is therefore a set of serialized states ready to be transferred to other parts of the system. Presignature pools for the online nodes are transferred to the runtime environment, while the offline node receives both the presignature pool and `ppd`. Key shares remain stored locally as a backup and as the basis for future presignature regeneration. For the example in Figure 6, where three nodes are used, `node_1` and `node_2` are transferred to PC2 for the runtime environment, together with the `public_key` directory for `btc_auxiliary`. `node_0` is transferred to the offline node. The `key_shares` directory remains in the initialization output for future generation.

This model is also practical and advantageous from a security perspective: the runtime environment does not need to contain long-term key shares, only consumable presignatures intended for a specific number of future signatures.

5.5. Serialization

The project uses the `rpm-serde` library for serialization, using the `MessagePack` format. This choice was motivated mainly by two reasons. First, `rpm-serde` provides a stable and commonly used way to serialize data structures in Rust through `serde` into a compact binary format. Second, at the time of implementation, the `bincode` library, which had long been considered a standard choice for binary serialization in the Rust ecosystem, was no longer maintained.

Using `MessagePack` is also practical for our use case. Compared with a text format such as JSON, it provides a more compact binary representation of data, which is advantageous when storing and transferring node state, especially presignature pools and other structures used during initialization. Since serialized data are

transferred between parts of the system using files, priorities were simple integration, reliable deserialization, and reasonable space efficiency. rmp-serde satisfies these requirements.

5.6. Coordinator Runtime

The runtime part of the system simulates how an MPC signing infrastructure could work in a real multi-device deployment. In the prototype, online nodes are represented as asynchronous tasks, while the central control point is the coordinator, which is itself an asynchronous task. The coordinator receives input from other parts of the system, especially `btc_auxiliary`, distributes signing requests to the online nodes, and creates the resulting signature after collecting their responses.

When the runtime environment starts, the coordinator first creates $n-1$ asynchronous tasks representing the online nodes. Communication between the coordinator and these nodes takes place locally through `mpsc::unbounded_channel`. The nodes do not communicate with one another; from the perspective of the runtime environment, it is sufficient for each node to receive commands from the coordinator and return its result. This architecture corresponds to the fact that, during runtime, the interactive MPC protocol in its original sense is no longer running; instead, nodes simply consume pre-generated presignatures locally.

Each online node supports two basic commands represented by an enum: `Init` and `Sign`. The `Init` command loads the local node state from its corresponding directory and initializes it. In practice, this means loading the serialized `presig_pool` into memory. After successful initialization, the node returns its pool size to the coordinator, allowing the coordinator to verify that all online nodes were loaded correctly and have the same number of presignatures. A simplification of the prototype is that if initialization fails, the entire process is repeated for all nodes at once; fine-grained recovery of a single node is not supported in the demo.

After successful initialization, the online nodes are ready to sign. The `Sign` command causes the coordinator to send the signing payload to every node. The node then consumes one presignature from its local pool and returns its `PartialSignature` to the coordinator. At that point, the coordinator has $n-1$ online partial signatures. After receiving the missing partial signature from the offline node, it performs the final aggregation and stores the resulting signature in the output directory.

The runtime component also supports the practical scenario in which the presignature pool becomes exhausted. During initialization, the coordinator determines the pool size of all online nodes and stores this value. It then tracks the number of signatures already performed and can detect when the pool is exhausted. At that point, new presignatures must be generated in the initialization program, transferred to the runtime environment, and `Init` must be run again to load the new node state. The system then returns to a state ready for further signing.

An important implementation detail of the runtime component is the `--prehashed` parameter. It determines whether the input payload should be considered already hashed data or whether hashing should be performed inside the MPC system. Without this parameter, the standard path is `DataToSign::digest::<Sha256>(payload)`, meaning that the payload is hashed first. With the auxiliary layer, however, the MPC system is not given arbitrary input data but a specific signing payload derived from a Bitcoin transaction, which is already a hash from the perspective of ECDSA. Hashing it again inside the MPC system would therefore produce an incorrect signature. The `--prehashed` parameter makes it possible to process the payload directly as a prehashed input without additional hashing. Conversely, during testing with random data or in a general payload mode, the parameter is not used and the hash is created inside the MPC system.

5.7. Offline Node

The offline node is a separate program intended to run on a device that is not connected to the Internet or to the online part of the system. Its purpose is to represent the final, manually operated signing party in the n -of- n model and thereby create an additional security barrier against compromise of the online infrastructure.

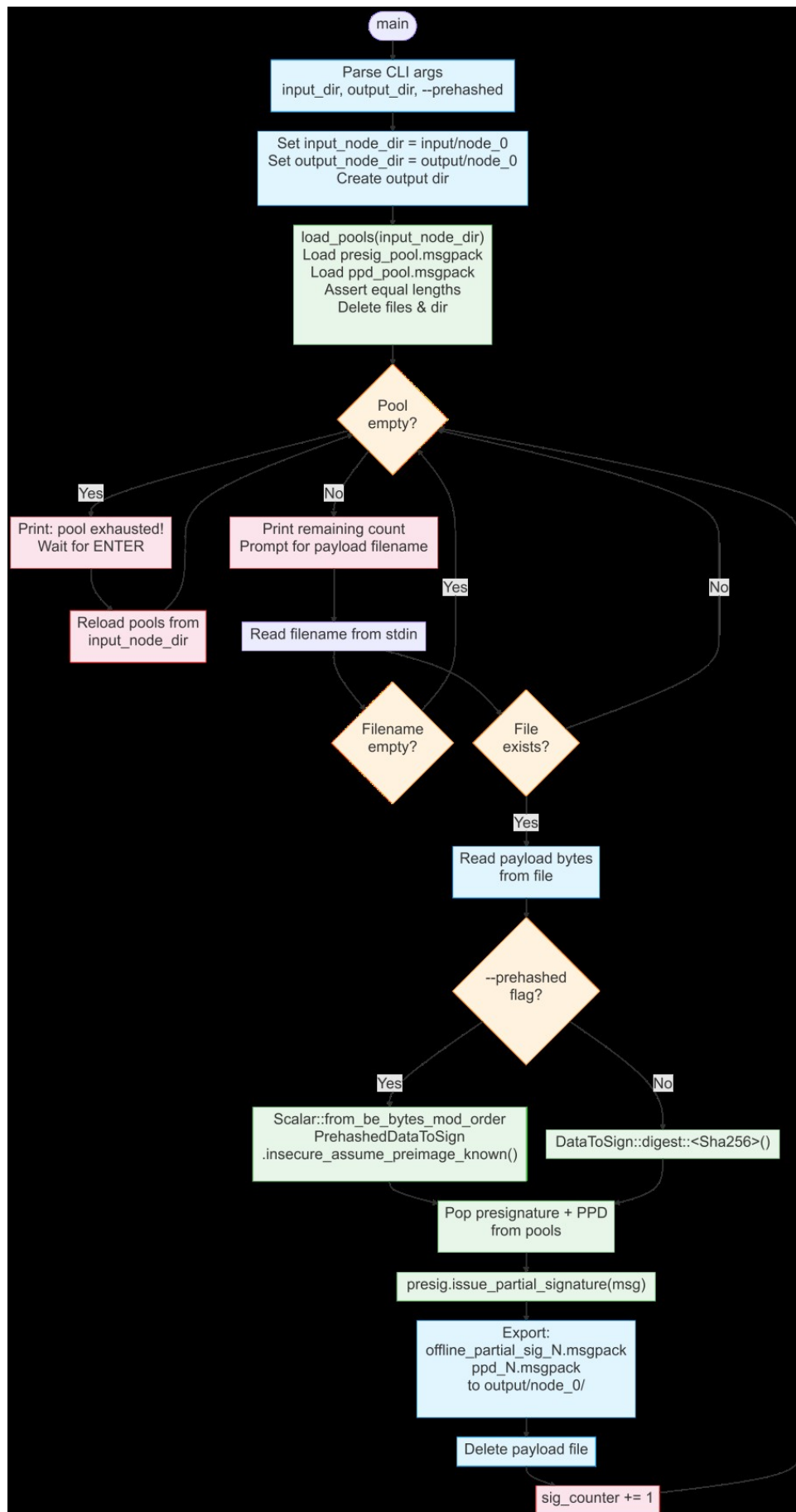
In terms of behavior, the offline node works similarly to the online runtime nodes: it also uses a local `presig_pool`, from which it consumes one presignature during signing and creates a `PartialSignature` for the supplied payload. Unlike the online nodes, however, it does not communicate with the coordinator through an internal communication channel; it is controlled manually through input and output files.

When the offline node starts, it loads the local `presig_pool` prepared during initialization. In addition, a separate FIFO structure containing `PresignaturePublicData` (ppd) is prepared and stored for the offline node. These data are needed for later signature aggregation because the final signature is created using the function:

```
combine(partial_signatures, commitments, m)
```

where the `commitments` parameter corresponds to the `PresignaturePublicData` associated with the presignature set that was used.

An important property of the offline node is its manual operation. The payload is delivered to it through the offline channel (in the prototype, through files transferred via a USB medium), the user starts the signing step locally on the offline device, and the result is then transferred back to the runtime environment through the same offline channel. This ensures that the final missing share of the signature is not created within the online infrastructure and that its creation requires physical interaction with the isolated device.



6. Expected Demonstration Flow

The purpose of the demonstration is to show end-to-end signing in an MPC/TSS model with one offline node. The demonstration is designed as a system divided into three main parts - initialization, runtime environment, and offline signing - complemented by an auxiliary layer that handles Bitcoin transactions. Only serialized data are transferred between these parts, either locally on one computer or through an offline channel. In the prototype, this offline channel is file transfer using a USB medium.

This chapter describes the expected demonstration flow at the level of the concrete implementation, rather than only as a general architectural proposal. The goal is to show how the system is initialized, how the data required for signing are prepared, how communication between the coordinator and online nodes takes place, and how the offline node becomes part of the process.

6.1. Basic Assumptions

The signing mode used is n-of-n (non-threshold), meaning that partial signatures from all participating nodes are required to create the final signature. The number of online nodes is variable in the prototype because they are represented as asynchronous Tokio tasks. There is always exactly one offline node, representing a separate, manually operated signing party.

Priebeh demonštrácie

MPC/TSS podpisovanie s jedným offline uzlom

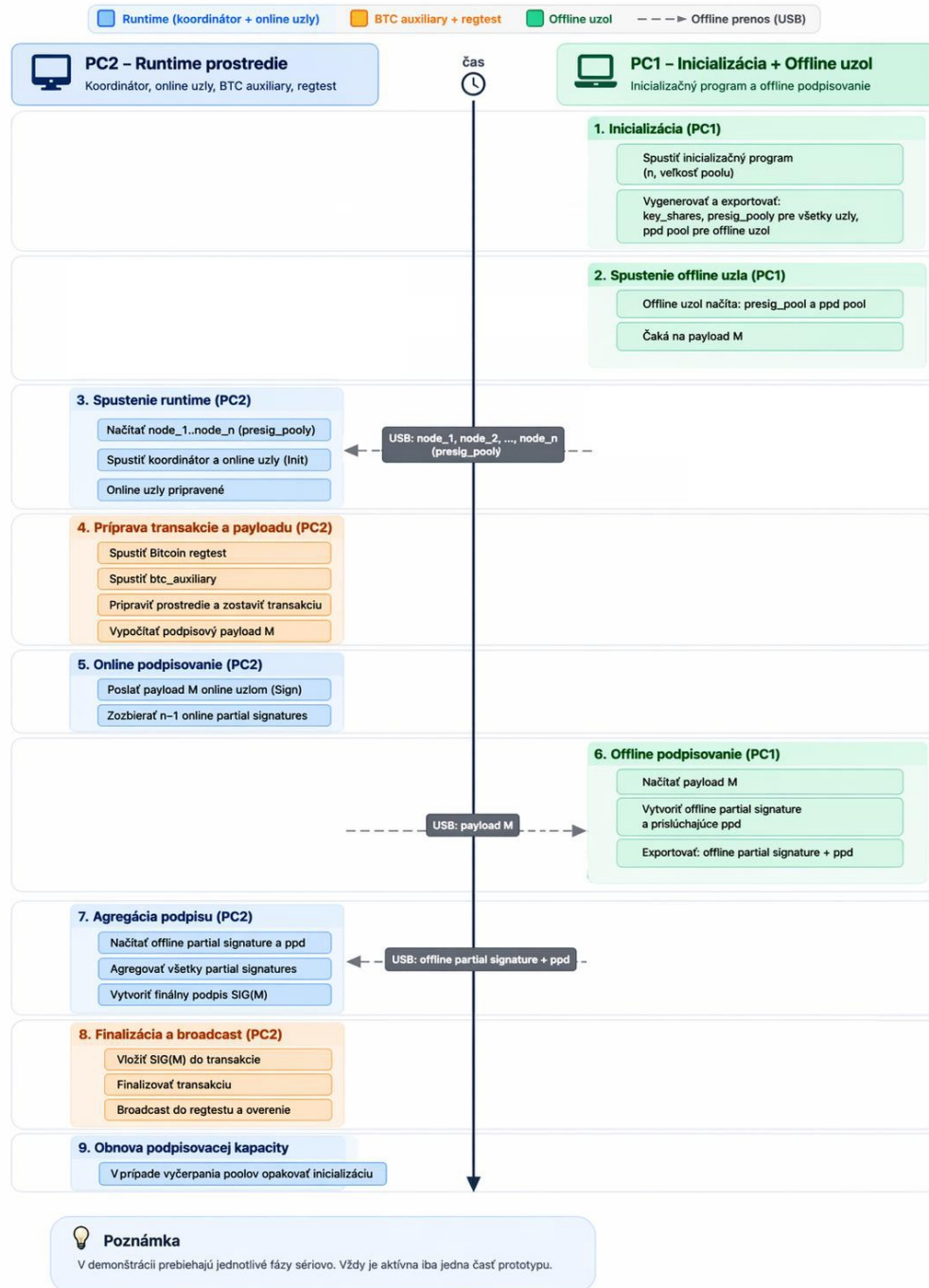


Figure 8: Demonstration flow

6.2. Procedure

0. Demonstration setup

0.1 Prepare two computers:

- PC1: initialization environment + offline node program
- PC2: runtime environment with coordinator + auxiliary layer + Bitcoin regtest

0.2 Prepare the offline transfer channel (in the demo, files on a USB medium).

1. Initialization phase (PC1)
 - 1.1 Start the initialization program in the isolated environment with the following parameters:
 - number of nodes
 - presignature pool size
 - 1.2 Check whether stored key_share values already exist:
 - if yes: reuse them
 - if no: generate them from scratch
 - 1.3 If the key shares are generated from scratch, perform:
 - generation of aux_info
 - generation of incomplete_keyshare
 - combination of aux_info + incomplete_keyshare -> key_share
 - 1.4 Then generate for all nodes:
 - FIFO presignature pools
 - for the offline node, also a FIFO set of ppd (PresignaturePublicData)
 - 1.5 Serialize the resulting data to files:
 - create a node_i directory for each node
 - store the corresponding presig_pool in each node_ directory
 - treat node_0 as the offline node
 - store the ppd FIFO structure additionally in node_0
 - store all key_share values separately in the key_shares directory
 - 1.6 Keep the initialization component available for possible future presignature generation.
2. Start the offline node (PC1)
 - 2.1 Start the offline node program.
 - 2.2 The offline node loads from its directory:
 - presig_pool
 - FIFO structure containing ppd
 - 2.3 The offline node enters a state in which it waits for a payload to sign.
 - 2.4 The offline node does not communicate over the network, only through:
 - input and output files
 - the offline transfer channel (USB)
3. Start the runtime environment (PC2)
 - 3.1 Transfer the exported states of the online nodes from PC1 to PC2 via USB:
 - node_1, node_2, ..., node_n directories
 - 3.2 Start the runtime environment on PC2.
 - 3.3 The coordinator:
 - creates n-1 online nodes as Tokio tasks
 - sends them the Init command
 - 3.4 Each online node, after receiving Init:
 - loads its presig_pool
 - initializes itself
 - returns the size of its pool to the coordinator
 - 3.5 The coordinator verifies:
 - that all online nodes were initialized successfully
 - that all pools have the same size
4. Start the Bitcoin component (PC2)
 - 4.1 Start the local Bitcoin blockchain in regtest mode.
 - 4.2 Start btc_auxiliary.

4.3 The auxiliary layer:

- connects to regtest
- prepares the local blockchain environment
- creates addresses and test conditions
- constructs a specific SegWit transaction
- computes the signing payload M from it

4.4 The auxiliary layer passes payload M to the coordinator.

5. Online signing (PC2)

5.1 The coordinator distributes payload M to the online nodes using the Sign command.

5.2 Each online node:

- takes one presignature from its local pool
- creates a partial signature for payload M
- sends the partial signature to the coordinator

5.3 The coordinator collects n-1 online partial signatures and enters the "offline signature missing" state.

6. Offline signing (PC1)

6.1 Transfer payload M from PC2 to PC1 via the offline channel (USB/file).

6.2 The offline node:

- loads payload M to be signed
- consumes one presignature from the local pool
- takes the corresponding ppd from the FIFO structure
- creates the missing partial signature for the offline node
- stores the result in a file
- stores the commitment (ppd) for this presignature set in a separate file

6.3 Transfer the resulting partial signature together with the ppd back to PC2.

7. Signature aggregation (PC2)

7.1 The coordinator loads:

- the offline PartialSignature
- the corresponding ppd

7.2 The coordinator performs aggregation:

- adds the offline partial signature
- combines the n partial signatures
- uses the ppd
- creates the final signature SIG(M)

7.3 The coordinator stores the resulting signature in the output directory.

8. Transaction finalization and submission to the blockchain (PC2)

8.1 The auxiliary layer:

- inserts signature SIG(M) back into the BTC transaction
- finalizes the transaction
- sends it to the local Bitcoin regtest

8.2 It is then verified that the blockchain accepted the transaction.

9. Recovery of signing capacity

9.1 The coordinator keeps track of:

- the pool size at initialization
- the number of signatures already performed

9.2 If the pool is exhausted:

- new presignatures are generated in initialization on PC1
- they are exported into the node directories
- they are transferred to the runtime environment

- Init is run again

9.3 The same procedure is applied to the offline node.

9.4 The system returns to a state ready for further signing.

Note (presignatures): After the initial initialization and generation of presignatures, the initialization component continues to run indefinitely as a "source" for later presignature replenishment. If the runtime environment consumes its presignatures, the coordinator requests that new presignatures be generated in the initialization component and the new presignatures are then exported again (in the demo through files/USB) to the runtime environment and the offline node. This is a deliberate limitation of the prototype and its design: replenishing presignatures requires a return to the initialization environment. In practice, a large number of presignatures can be generated in advance, significantly reducing the need for replenishment; at the same time, designing this mechanism in a more robust and efficient way remains an open area for future improvement.

6.3. Correctness Verification

The demonstration is considered successful if:

- the coordinator receives partial signatures from all online nodes
- the offline node creates and returns its partial signature together with the corresponding ppd
- the coordinator successfully aggregates the final signature
- the auxiliary layer finalizes the transaction
- the resulting transaction is accepted by Bitcoin regtest

7. User Guide

Available in the GitHub repository of the prototype. Because the repository is private, access has to be granted; this will of course be possible for the purposes of the university project.

8. Threat Model

In this prototype, we consider an attacker attempting to gain control of digital assets either by compromising software and infrastructure (the online environment) or through physical access to devices. The fundamental motivation for using MPC/TSS is to eliminate the single point of failure (SPOF) in private-key management: no node holds the complete private key in one place, and therefore compromising a single machine by itself should not result in key exfiltration.

8.1. Coordinator Compromise (SPOF at the Authorization Level)

Although MPC distributes the risk of private-key leakage, the coordinator remains a critical component from the perspective of signing authorization. The coordinator is the point that receives signing requests (for example, transactions) and distributes the payload to the MPC network for signing. If an attacker compromises the coordinator, they can initiate signing of arbitrary data and attempt to create a malicious transaction without directly compromising the signing nodes.

In the prototype, this risk is mitigated by introducing one offline node whose participation is required in the n-of-n mode to create the final signature. The offline node requires manual operation: the payload to be signed is transferred through an offline channel (for example, as a file on a USB medium), manually inspected on the offline device, and only then is a partial signature generated. If the payload does not correspond to an authorized request, for example because the transaction is unexpected, the address does not satisfy whitelisting/blacklisting rules, or it does not match the user request, the offline node refuses to sign. This means that compromising the coordinator alone is not sufficient to steal the funds.

8.2. Why a Single Offline Signing Device Without MPC Is Not Enough

It might seem that if the offline node serves as an authorization "brake", a single offline device containing the complete private key would be sufficient. Such a solution, however, creates a new single point of failure: if the offline device is physically compromised, an attacker would gain full control over all funds. Combining online MPC with an offline node distributes the risk: an attacker must simultaneously carry out a successful attack on the online infrastructure (to initiate and coordinate signing) and a physical attack (to obtain control of the offline

part of the process). This combination significantly increases the difficulty of an attack compared with a model relying on a single device.

8.3. Placement of PPD and Its Security Implications

In CGGMP24, partial signatures alone are not sufficient for final signature aggregation; the combine function also requires the corresponding PresignaturePublicData (ppd). These data are bound to a particular presignature set, and without them the final signature cannot be correctly constructed.

For one specific presignature set, the ppd is identical for all nodes. It therefore does not make sense to store it redundantly at every online node together with its local presig_pool, since that would only unnecessarily increase the volume of transmitted and stored data.

In our prototype, ppd is therefore stored with the offline node. This decision is based on the fact that the initialization phase and the offline node run on the same trusted device. ppd therefore does not have to be exported into the runtime environment together with the online states; it is sufficient for the offline node to return it to the coordinator together with the corresponding partial signature. The coordinator then uses the ppd during final signature aggregation.

This approach is more memory-efficient, simplifies state-data management, and does not increase the attack surface of the prototype's model. In another deployment, ppd could be stored elsewhere as long as it is available at the time of aggregation and the required security properties of the environment are preserved.

9. Conclusion

Work on this project produced interesting results and was a valuable experience from both academic and practical perspectives. Solving the problem required combining theoretical research in MPC/TSS protocols with the design of a realistic architecture and the subsequent implementation of a functional prototype. The project was therefore not merely an implementation exercise, but also a process of progressively refining the model, validating design decisions, and finding a reasonable balance between security, simplicity, and practical usability.

We fulfilled all essential requirements for TSS signing: the key is divided into N parts stored on independent nodes, no node possesses the complete private key, and the signature is created through cooperation between multiple parties. The prototype is also designed to be multichain within the intersection of ECDSA over secp256k1, making it usable for Bitcoin (legacy/SegWit) and Ethereum, with potential for future extensions depending on the capabilities of the selected signature scheme.

We also architecturally addressed the requirements placed on the signing protocol itself. Private keys are never transferred between nodes; only partial values required to create the signature are exchanged. The design supports both online and offline signing - the offline node is manually operated in the model and signs requests outside the online environment. The result is a standard single-signature that appears on the blockchain in the same way as a signature created by a single private key, without a recognizable multisig trace.

In addition, the originally supplementary auxiliary layer was successfully implemented, making it possible to demonstrate the prototype on a real, although local, Bitcoin infrastructure in regtest mode. The project therefore goes beyond signing random data and demonstrates the correct operation of the complete end-to-end flow from transaction construction, through signing-payload preparation and MPC signing, to transaction finalization and acceptance by the blockchain.

Compared with software projects in general, this is probably a relatively small project in terms of total lines of code. We do not consider this a disadvantage, but rather a natural consequence of the type of problem being solved. In cryptography, security, and auditability, it is desirable to aim for the simplest and clearest solution that minimizes unnecessary complexity. Just as a significant portion of the effort was devoted to researching the problem, part of the effort was also spent refactoring and simplifying the implementation so that the resulting code would be as readable, compact, and auditable as possible. From this perspective as well, we consider the chosen result appropriate for the goals of the project.

In conclusion, the resulting implementation meets the goals defined in the project documentation and represents a consistent, functional, and further extensible foundation for MPC/TSS signing in a custodial environment.

9.1. Further Directions

Although the final prototype meets the main goals and demonstrates functional MPC/TSS signing, including an offline node and a Bitcoin auxiliary layer, it naturally opens several possibilities for further development.

The most important area for further development is signing authorization. MPC itself handles key distribution and reduces the risk of key compromise, but by itself it does not decide which signing requests are legitimate. A production deployment therefore needs an additional mechanism above the coordinator that enforces signing rules or some form of multi-level approval.

Another natural direction is more robust management of presignatures and state data. In the prototype, we deliberately chose a simple FIFO-pool model because it is easy to understand for demonstration purposes and matches the expected control flow well. In the future, it would be appropriate to add safer presignature management, detection of desynchronization in structures working with presignatures, recovery after failures, or an entirely different approach using addressable presignature-set management through explicit identifiers, such as a hash map or some form of database.

Significant room for further development also exists in moving from the locally simulated runtime model to genuinely separated nodes. In the current prototype, online nodes are represented as asynchronous tasks without encrypted communication and without comprehensive recovery mechanisms. A production version would have to address all of this, including nodes represented by separate processes or devices, authenticated communication between components, monitoring, and resilience to failures.

The offline node and its workflow also represent a separate area for development. In the prototype, offline interaction is implemented using files and a USB medium, which is suitable for demonstration purposes. In the future, a more robust request/response data format or alternative offline channels, such as the QR codes already mentioned, could be considered.

A natural continuation would also be to expand the auxiliary layer. In the current implementation, it demonstrates a Bitcoin SegWit scenario in a regtest environment, providing practical evidence that the entire prototype works. In the future, it could be extended with an auxiliary layer for Ethereum or other cryptocurrencies, making the multichain character of the proposed MPC core even clearer.

Finally, from a security and operational perspective, it would be appropriate to add mechanisms that are deliberately simplified in the prototype, such as encryption of stored data, integrity verification of exports and imports, secure deletion of consumed presignatures from disk, and more extensive system testing.

Based on this, the final prototype should not be viewed as a closed solution, but as a functional and verified foundation on which further work can be built. In this sense, the project fulfills not only a demonstration goal but also a research-and-practical goal: it shows that the proposed model works while also creating room for further technical and security development.